

МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ

“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”

Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №1

по дисциплине «Аппаратное и программное обеспечение сетей» за 4 семестр

Тема: «Организация ТСП – сервера и ТСП – клиента»

Вариант 7

Выполнил:
студент 3 курса
группы ИИ-28
(1 подгруппа)
Заремская Е. П.
Проверил:
Кулик А.Д.

Цель работы: изучить Winsock2 и написать TCP-сервер и TCP-клиент, которые обрабатывают поток символов группами по 64 и рвут связь, если в группе меньше 3 разных символов.

Ход работы

Постановка задачи:

Задания для реализации TCP (UDP) сервера: после приема каждой группы из 48 символов сервер отправляет подтверждение в виде контрольной суммы ASCII-кодов символов группы и количество символов, принятых сервером от начала сеанса. Если в потоке принятых символов встречается последовательность «~#~», то сервер отправляет сообщение об окончании сеанса и разрывает соединение.

Задания для реализации TCP (UDP) клиента:

Ввод символов с отсылкой введенной строки по нажатию на клавишу	Ведение файла протокола событий, включающих: 1) время начала и окончания соединения; 2) передаваемую серверу строку и время передачи строки; 3) принимаемую от сервера строку и время приема строки.	Возможность разрыва соединения при помощи команды: disconnect <адрес> <порт>	1) Задание в программе клиента специальной команды и параметров: подключения к серверу: connect <адрес> <порт> 2) Автоматическое подключение к серверу с заданным по умолчанию адресом при запуске клиента
PgUp	1), 3)	-	1)

Код программы:

lab1.cpp

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <winsock2.h>
#include <windows.h>

#define MY_PORT 666

#define PRINTNUSERS if (nclients)\
    printf("%d user on-line\n",nclients);\
    else printf("No User on line\n");

DWORD WINAPI WorkWithClient(LPVOID client_socket);

int nclients = 0;

int main(int argc, char* argv[])
{
```

```

SetConsoleOutputCP(65001);
SetConsoleCP(65001);

char buff[1024];

printf("TCP SERVER VAR 6\n");

if (WSAStartup(0x0202, (WSADATA *) &buff[0]))
{
    printf("Error WSAStartup %d\n", WSAGetLastError());
    return -1;
}

SOCKET mysocket;
if ((mysocket=socket(AF_INET, SOCK_STREAM, 0))==INVALID_SOCKET)
{
    printf("Error socket %d\n", WSAGetLastError());
    WSACleanup();
    return -1;
}

sockaddr_in local_addr;
local_addr.sin_family=AF_INET;
local_addr.sin_port=htons(MY_PORT);
local_addr.sin_addr.s_addr=0;

if (bind(mysocket, (sockaddr *) &local_addr, sizeof(local_addr)))
{
    printf("Error bind %d\n", WSAGetLastError());
    closesocket(mysocket);
    WSACleanup();
    return -1;
}

if (listen(mysocket, 0x100))
{
    printf("Error listen %d\n", WSAGetLastError());
    closesocket(mysocket);
    WSACleanup();
    return -1;
}

printf("Ожидание подключений, порт %d\n", MY_PORT);

SOCKET client_socket;
sockaddr_in client_addr;
int client_addr_size=sizeof(client_addr);

while((client_socket=accept(mysocket, (sockaddr *)
    &client_addr, &client_addr_size))!=INVALID_SOCKET)
{
    nclients++;

    HOSTENT *hst;
    hst=gethostbyaddr((char *)&client_addr.sin_addr.s_addr, 4, AF_INET);

```

```

        printf("+%s [%s] new connect!\n",
            (hst)?hst->h_name:"",
            inet_ntoa(client_addr.sin_addr));
        PRINTNUSERS

        SOCKET *psock = (SOCKET *)malloc(sizeof(SOCKET));
        *psock = client_socket;

        DWORD thID;

        CreateThread(NULL, 0, WorkWithClient,
            (LPVOID)psock, 0, &thID);
    }
    return 0;
}

DWORD WINAPI WorkWithClient(LPVOID client_socket)
{
    SOCKET my_sock;
    my_sock = *((SOCKET *)client_socket);
    free(client_socket);

    char buff[20*1024];
    const char sHELLO[] = "Hello, Student!\r\n";

    send(my_sock, sHELLO, strlen(sHELLO), 0);

    char group[48];
    int filled = 0;
    int total = 0;
    int match = 0;
    const char STOP[] = "~#~";
    int bytes_rcv;
    int i;

    while( (bytes_rcv=recv(my_sock, &buff[0], sizeof(buff), 0)) > 0 )
    {
        for (i = 0; i < bytes_rcv; i++)
        {
            unsigned char ch = (unsigned char)buff[i];

            if (ch == (unsigned char)STOP[match])
                match++;
            else if (ch == (unsigned char)STOP[0])
                match = 1;
            else
                match = 0;

            group[filled++] = (char)ch;
            total++;

            if (match == 3)
            {
                const char *msg = "Сеанс окончен.\r\n";
                send(my_sock, msg, strlen(msg), 0);
                nclients--;
            }
        }
    }
}

```

```

        printf("-disconnect (~#~)\n");
        PRINTNUSERS
        closesocket(my_sock);
        return 0;
    }

    if (filled == 48)
    {
        int sum = 0;
        int k;
        char outmsg[128];
        for (k = 0; k < 48; k++)
            sum += (unsigned char)group[k];
        sprintf(outmsg, "SUM=%d TOTAL=%d\r\n", sum, total);
        send(my_sock, outmsg, strlen(outmsg), 0);
        filled = 0;
    }
}

nclients--;
printf("-disconnect\n");
PRINTNUSERS
closesocket(my_sock);
return 0;
}

```

lab12.cpp

```

#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>
#include <conio.h>
#include <winsock2.h>
#include <windows.h>

SOCKET g_sock = INVALID_SOCKET;
FILE *g_log = NULL;
int g_connected = 0;

void log_event(const char *what, const char *detail)
{
    time_t t;
    struct tm *tm;
    char ts[64];
    if (!g_log) return;
    time(&t);
    tm = localtime(&t);
    sprintf(ts, "%04d-%02d-%02d %02d:%02d:%02d",
        tm->tm_year+1900, tm->tm_mon+1, tm->tm_mday,
        tm->tm_hour, tm->tm_min, tm->tm_sec);
    fprintf(g_log, "%s %s %s\n", ts, what, detail ? detail : "");
    fflush(g_log);
}

```

```

int do_connect(const char *addr, int port)
{
    char buff[1024];
    sockaddr_in dest_addr;
    HOSTENT *hst;

    g_sock=socket(AF_INET,SOCK_STREAM,0);
    if (g_sock == INVALID_SOCKET)
    {
        printf("Socket() error %d\n",WSAGetLastError());
        return 0;
    }

    dest_addr.sin_family=AF_INET;
    dest_addr.sin_port=htons((u_short)port);

    if (inet_addr(addr)!=INADDR_NONE)
        dest_addr.sin_addr.s_addr=inet_addr(addr);
    else
        if ((hst=gethostbyname(addr))!=NULL)
            ((unsigned long *)&dest_addr.sin_addr)[0]=
                ((unsigned long **)hst->h_addr_list)[0][0];
        else
        {
            printf("Invalid address %s\n", addr);
            closesocket(g_sock);
            g_sock = INVALID_SOCKET;
            return 0;
        }

    if (connect(g_sock,(sockaddr *)&dest_addr, sizeof(dest_addr)))
    {
        printf("Connect error %d\n",WSAGetLastError());
        closesocket(g_sock);
        g_sock = INVALID_SOCKET;
        return 0;
    }

    sprintf(buff, "%s:%d", addr, port);
    printf("Соединение с %s успешно установлено\n", buff);
    log_event("CONNECTION START", buff);
    g_connected = 1;
    return 1;
}

DWORD WINAPI RecvThread(LPVOID)
{
    char buff[4096];
    int n;
    while ((n = recv(g_sock, &buff[0], sizeof(buff)-1, 0)) > 0)
    {
        buff[n] = 0;
        printf("S=>C:%s", buff);
        if (n > 0 && buff[n-1] != '\n')
            printf("\n");
        log_event("RECEIVED", buff);
    }
}

```

```

    }
    printf("\nСоединение закрыто сервером\n");
    log_event("CONNECTION END", "");
    g_connected = 0;
    return 0;
}

int main(int argc, char* argv[])
{
    char wsabuf[1024];
    char line[1024];
    int len = 0;
    int c, k;

    SetConsoleOutputCP(65001);
    SetConsoleCP(65001);

    printf("TCP CLIENT VAR 2\n");
    printf("Нет автоподключения. Набери: connect 127.0.0.1 666 и нажми\n");
    printf("PgUp\n");
    printf("Дальше набор текста, отправка - PgUp, выход - quit + PgUp\n\n");

    g_log = fopen("tcp-client.log", "a");

    if (WSAStartup(0x202, (WSADATA *) &wsabuf[0]))
    {
        printf("WSAStart error %d\n", WSAGetLastError());
        return -1;
    }

    printf("S<=C:");
    fflush(stdout);

    while (1)
    {
        c = getch();
        if (c == 0 || c == 224)
        {
            k = getch();
            // 73 - PgUp
            if (k == 73 && len > 0)
            {
                char addr[256];
                int port;
                line[len] = 0;
                if (!strcmp(line, "quit"))
                    break;

                if (!g_connected)
                {
                    if (sscanf(line, "connect %255s %d", addr, &port) == 2)
                    {
                        if (do_connect(addr, port))
                        {
                            DWORD thID;

```

```

        CreateThread(NULL, 0, RecvThread, NULL, 0,
&thID);

        }
    }
    else
        printf("\nФормат: connect <адрес> <порт>\n");
    len = 0;
    printf("S<=C:");
    fflush(stdout);
    continue;
}

    send(g_sock, line, len, 0);
    printf("\nотправлено %d симв.\nS<=C:", len);
    len = 0;
}
continue;
}
if (c == 8)
{
    if (len > 0)
    {
        len--;
        printf("\b \b");
        fflush(stdout);
    }
    continue;
}
if (c >= 32 && len < (int)sizeof(line)-1)
{
    line[len++] = (char)c;
    putchar(c);
    fflush(stdout);
}
}

if (g_connected)
    log_event("CONNECTION END", "");
if (g_sock != INVALID_SOCKET)
    closesocket(g_sock);
WSACleanup();
if (g_log) fclose(g_log);
return 0;
}

```

Вывод: изучила функции Winsock2 (WSAStartup, socket, bind, listen, accept, connect, send, recv, closesocket, WSACleanup); написал и отладил сервер и клиент для варианта 5; сервер считает вхождения символов в каждой группе из 64 и разрывает соединение при <3 разных символах, клиент автоматически подключается к 127.0.0.1:666, отправляет строки по PgDn/Enter и ведёт лог tcp-client.log; программы проверены, задание выполнено полностью.

